

AI/ML Possibilities in OpenROAD

ECE 260C, Spring 2026
Andrew B. Kahng

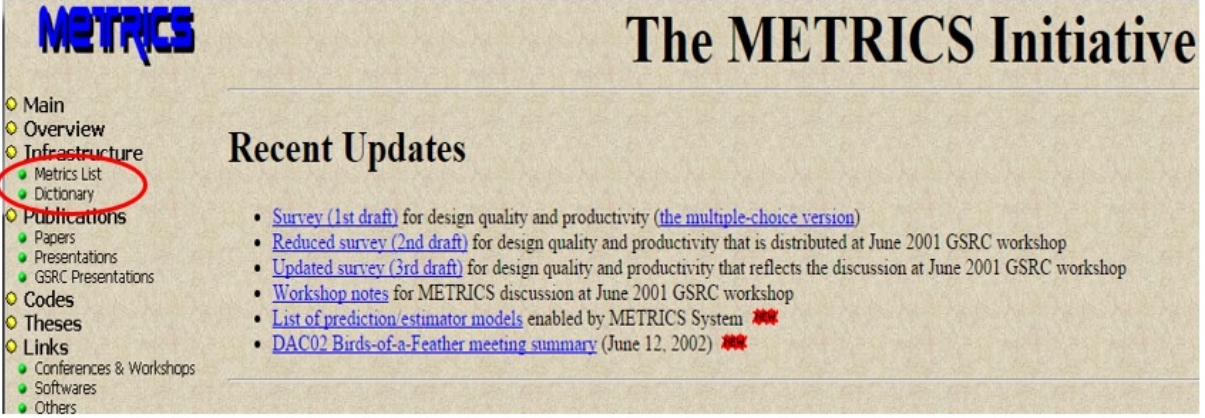
Agenda

- **Old School**

- Predict: Will RouteOpt finish with clean signoff, <1000 DRVs by tomorrow night?
- Classify: Out of these 50 floorplans + budgets, which 3 should go into trial SP&R?
- Estimate: How many hold buffers will the tool add into this post-CTS layout?
- Guide: What P&R tool setup/script will obtain the best QOR within next 36 hours?

- A. You need models to have predictions
- B. You need predictions to leverage in exploration
- C. What you can't predict, you guardband
- D. What you don't explore, you leave on the table
- E. C and D are bad for product quality and schedule

- “Catechism” has never changed.
- Record, Mine, Model, Predict/Act, Measure, Improve (iterate)

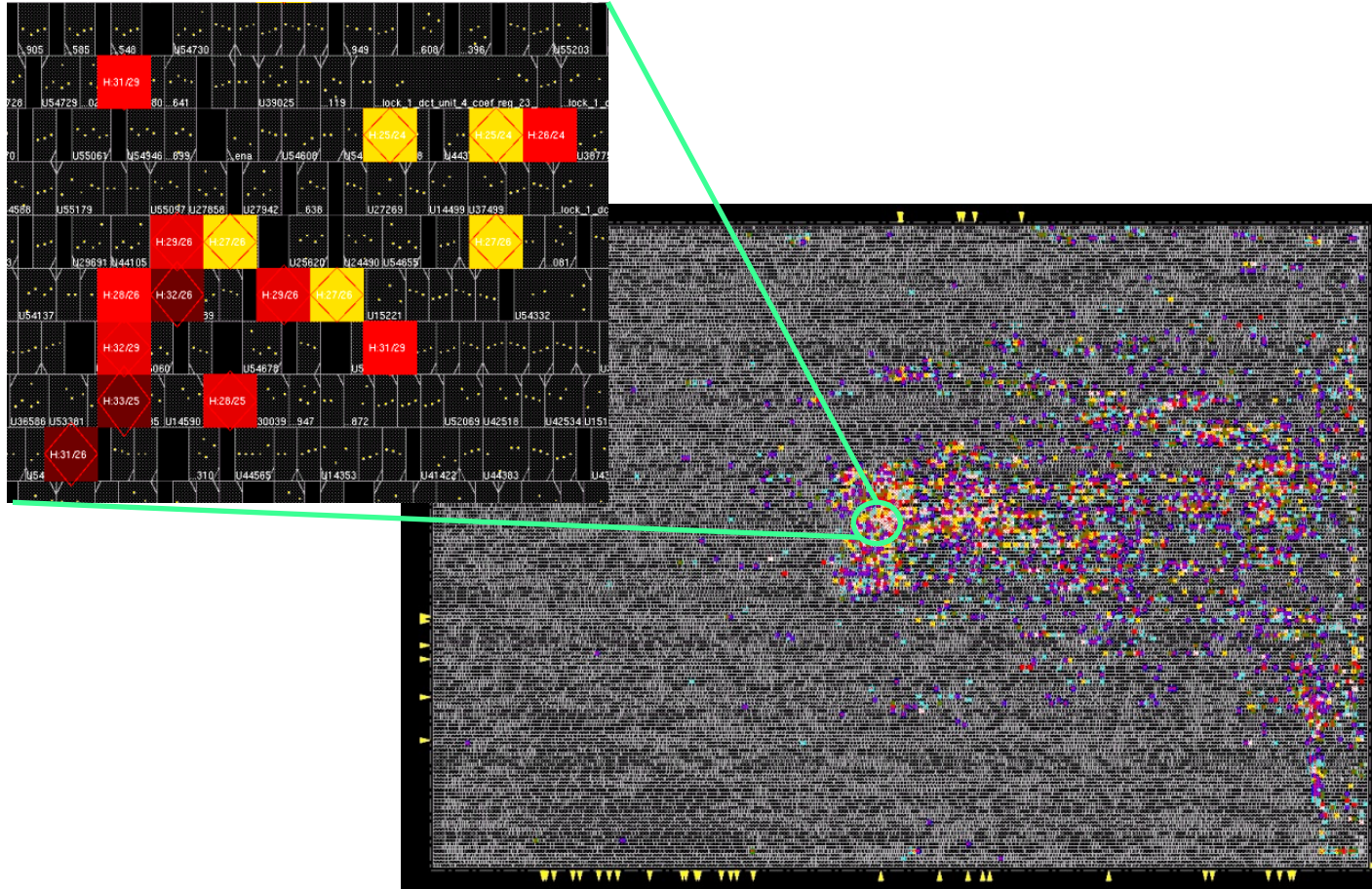


- METRICS (1999; DAC00, ISQED01):
“Measure to Improve”
 - Goal #1: Predict outcome
 - Goal #2: Find sweet spot (field of use) of tool, flow
 - Goal #3: Dial in design-specific tool, flow knobs

<http://vlsicad.ucsd.edu/GSRC/metrics>

Congestion Map from Trial (Global) Routing

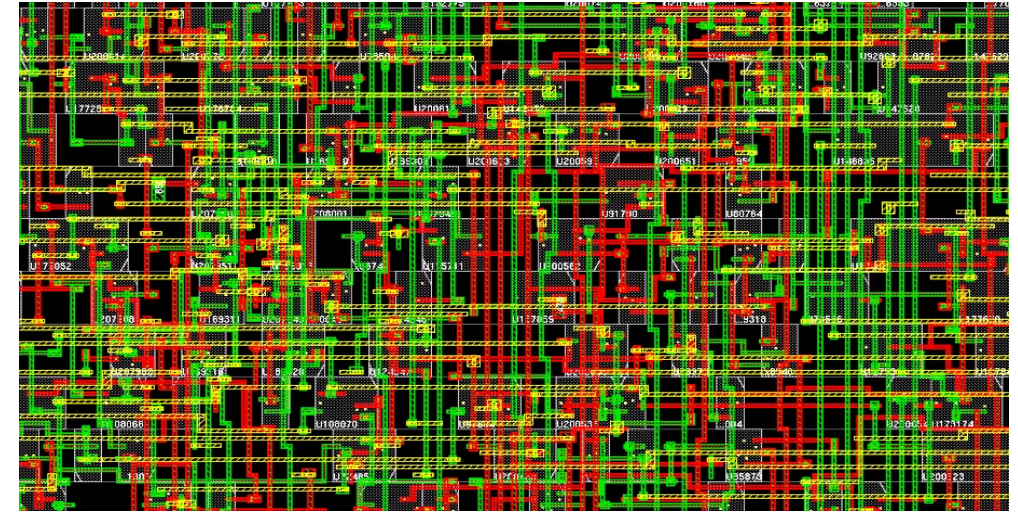
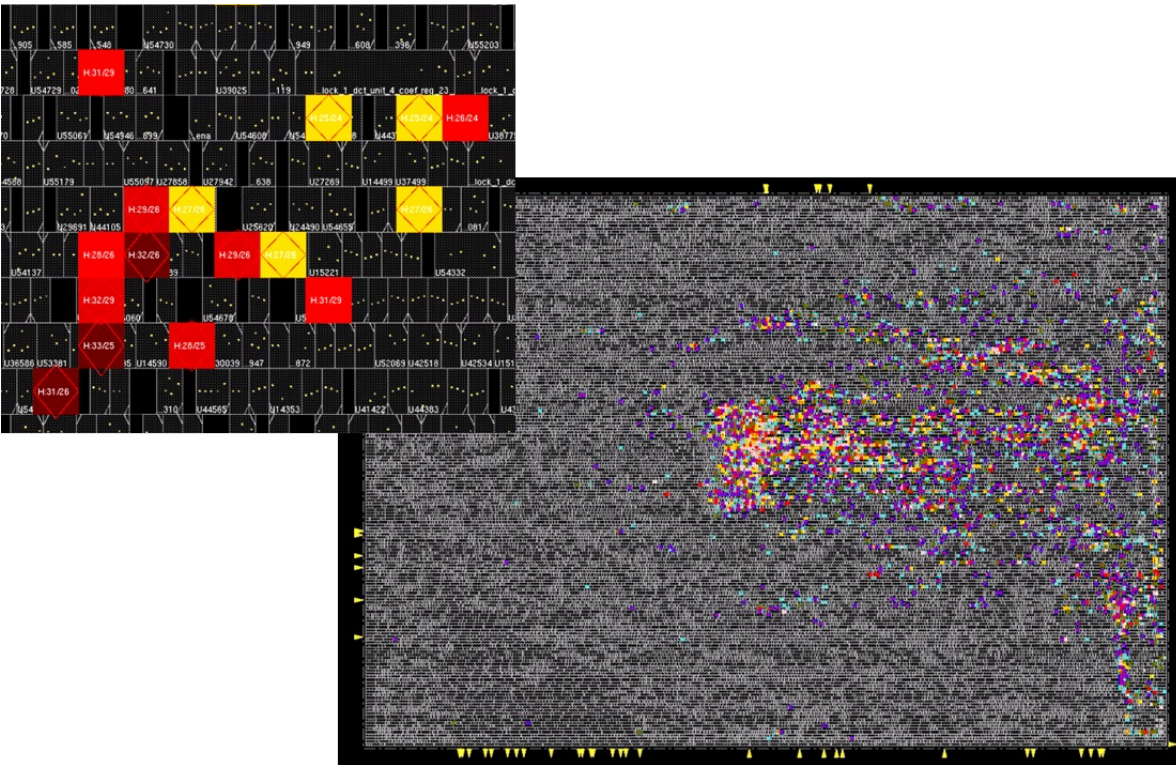
Remember: one of the three ways that we “predict the future” in IC design is by doing it under the hood → severe challenge of exactitude vs. efficiency



Prediction of Routing Outcomes

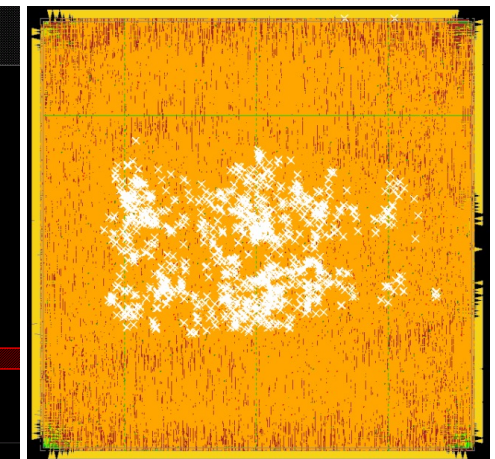
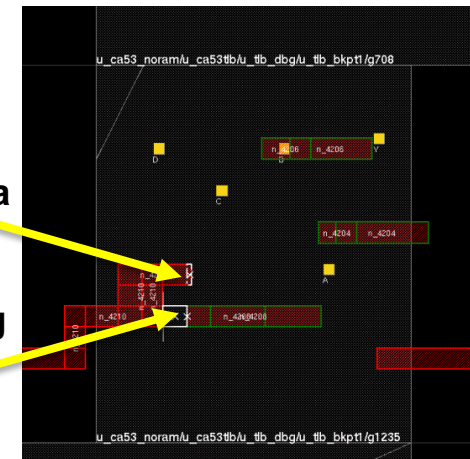
- Cost of false positives
- “BCWYAF”
- Usage of prediction

GR-based congestion map



Min-area
viol

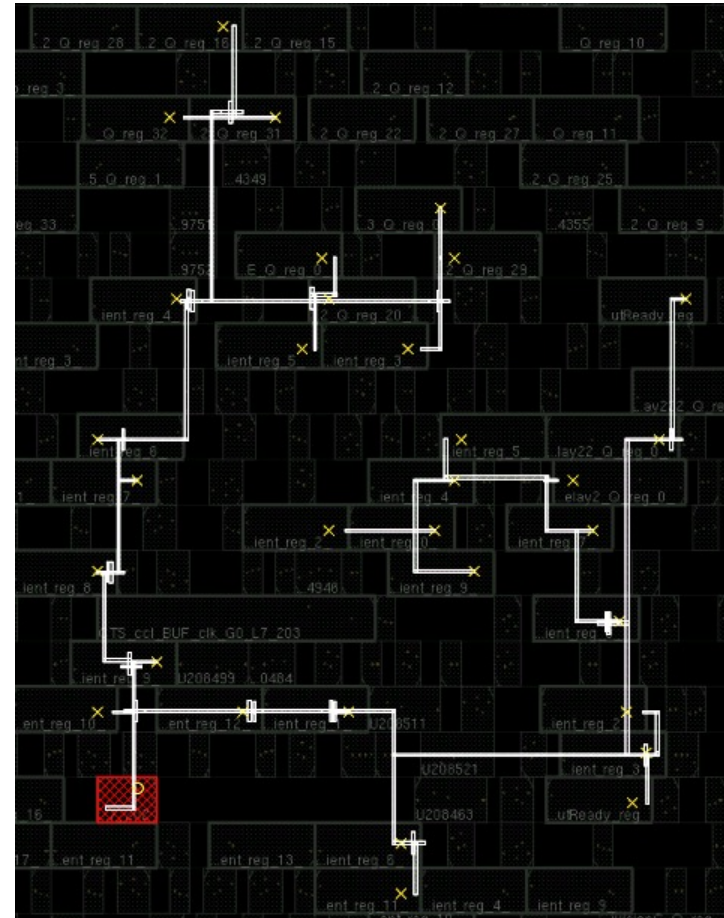
Spacing
viol



Post-DR DRVs

Bottom-Up Outcome Models (Example: Clock Tree Buffer + Fanouts)

- Red rectangle = **source pin** (clock buffer output)
- X's = **sink pins** (clock pins of flip-flops)
- **Rectilinear Steiner tree** over $n = 33$ terminals
- **Where should CTS place the buffer?**
 - Skew vs. WL vs. ...
 - This clock sub-net
- **What we really want is a very fast proxy for the CTS flow !**



More

- April 20: Virtual buffering and MLBuf
- Operator / transform sequencing
 - ABC technology mapping
 - Resizer optimizer

ML Opportunity 1: ML for Buffering

• Motivation

- Timing closure requires extensive buffer insertion
- Buffer insertion during the global placement process
 - Fix ERC violations
 - Give more accurate slack information
 - Consider the buffer area effect
- Actual buffer insertion slows down the GPU-accelerated placer **because the topology changes!**

→ Buffer-porosity-aware placement

→ Lightweight virtual buffering strategy

• Limitations of existing methods

- Traditional van Ginneken-L
- Computationally expensive
- Modern ML-based approach
 - Omit Electrical Rule Check
 - Fail to “close the loop”

UCSD ECE 260C SP26 Kahng

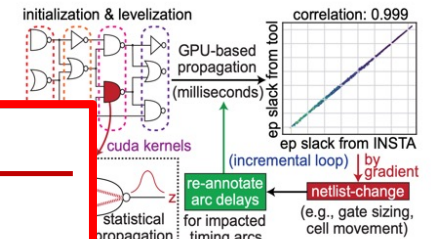
ML Opportunity 2: Fast Timing Prediction as STA Proxy

• Problem

- STA is the **runtime bottleneck** in timing-driven optimization
- Traditional STA tools are **sequential** and **CPU-bound**
- Each trial move triggers an incremental timing update
- For large-scale designs, thousands of STA calls dominate the runtime

• INSTA [2]

- A **fully differentiable** STA engine running on **GPU**
- Begins with a **one-time initialization** from a



Statistical Static Timing Analysis

31

ML Opportunity 3: ML for Sizing

• Problem

- Given a gate and its context, predict the optimized size variant directly

• Representative papers

• RL-Sizer [3]

- **GNN-powered** policy gradient **RL** framework
- State = circuit graph (encoded by GNN)
- Action = select a gate and choose a new size
- Reward = improvement in timing metrics (WNS/TNS)

• TransSizer [4]

- **Transformer** architecture for gate sizing
- Use self-attention to capture global dependencies across gates
- Directly predict optimal gate sizes for all gates in one forward pass

• LearnSize [5]

- **Multi-modal modeling** combining graph features and layout features
- Efficient gradient generation and back-propagation designed for large-scale circuits
- Physically-aware: considers placement constraints during sizing

[3] Y. -C. Lu, et al., "RL-Sizer: VLSI Gate Sizing for Timing Optimization using Deep Reinforcement Learning," Proc. DAC, 2021.

[4] S. Nath, et al., "TransSizer: A Novel Transformer-Based Fast Gate Sizer," Proc. ICCAD, 2022.

[5] Y. Ye et al., "Learning-Driven Physically Aware Large-Scale Circuit Gate Sizing," TCAD, 44(5)(2025), pp. 1901-1914.

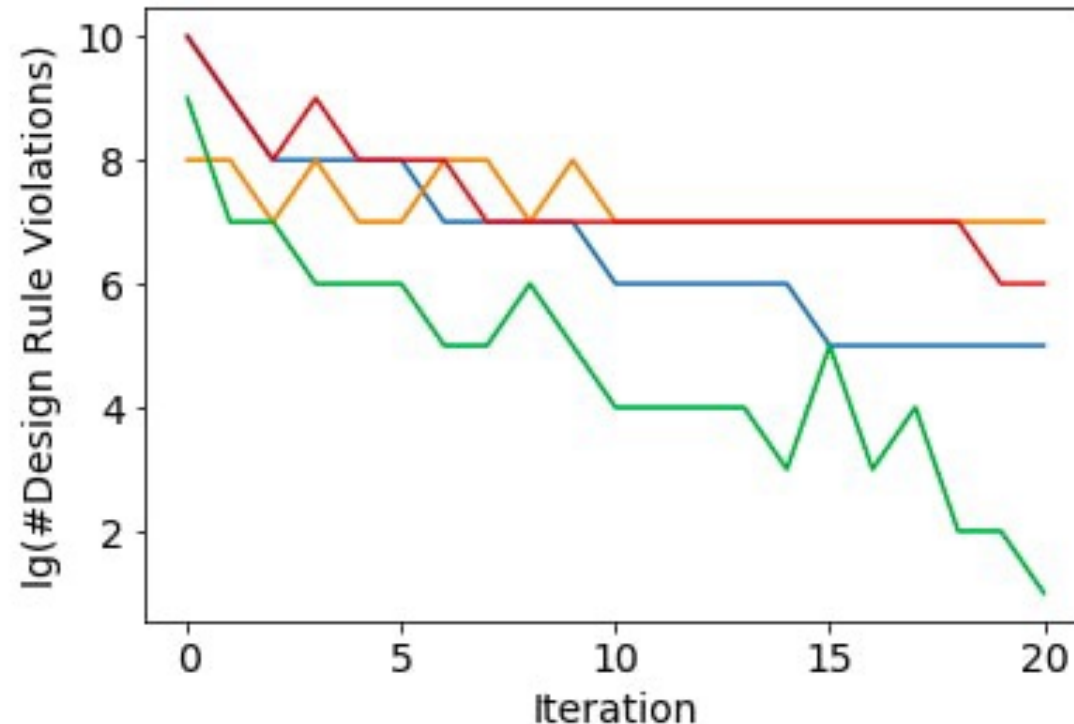
UCSD ECE 260C SP26 Kahng

32

6

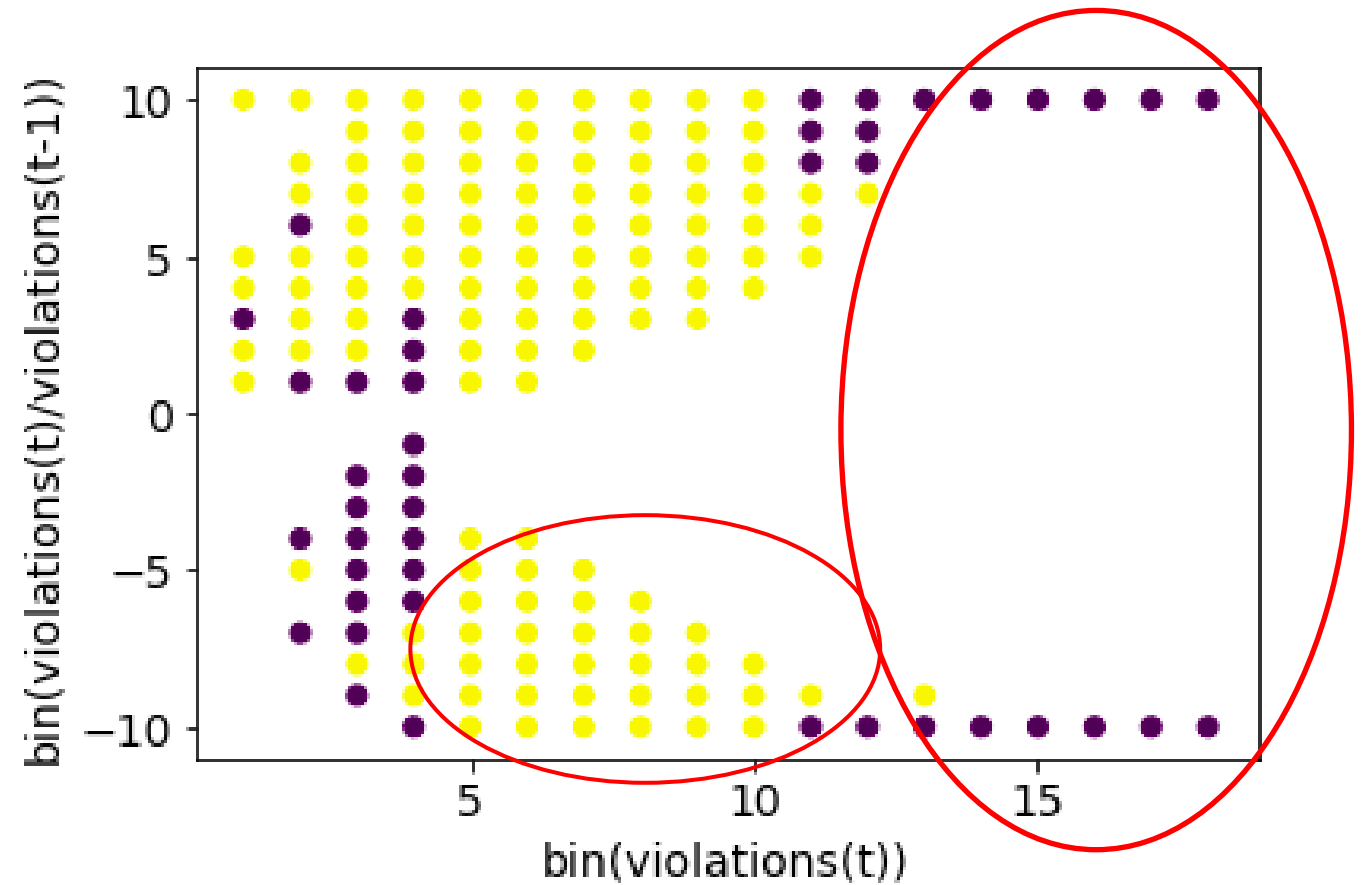
Leverage: Prediction of Doomed Runs?

- Which jobs will end up with too many post-route DRVs?
- Approach: track and project metrics as time series
- Markov decision process (MDP): terminate “doomed runs” early
- Shown: 4 example progressions of #DRVs (commercial router)
 - Stopping red, yellow runs early would save resources and schedule !

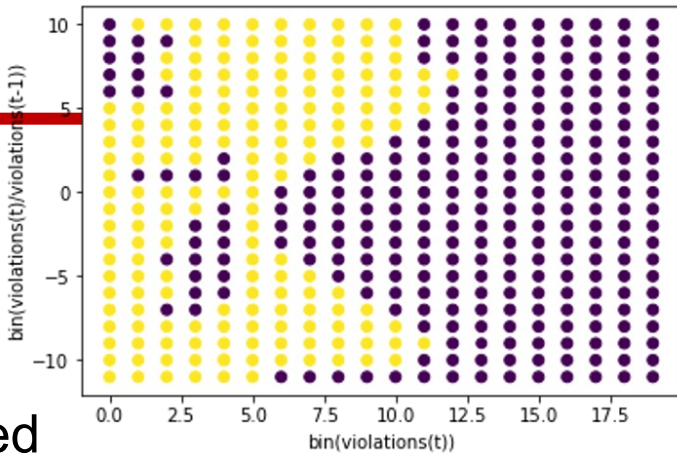


Markov Decision Process = “Strategy Card”

- **State space** from Fibonacci binning
- **Actions** – **GO** or **STOP**
- **Rewards** at each state – e.g., small negative reward for *non-stop* state, large positive reward for *stop* with low #DRVs, etc.
- Automatically trained MDP
“strategy card”: **Yellow = GO**, **Purple = STOP**



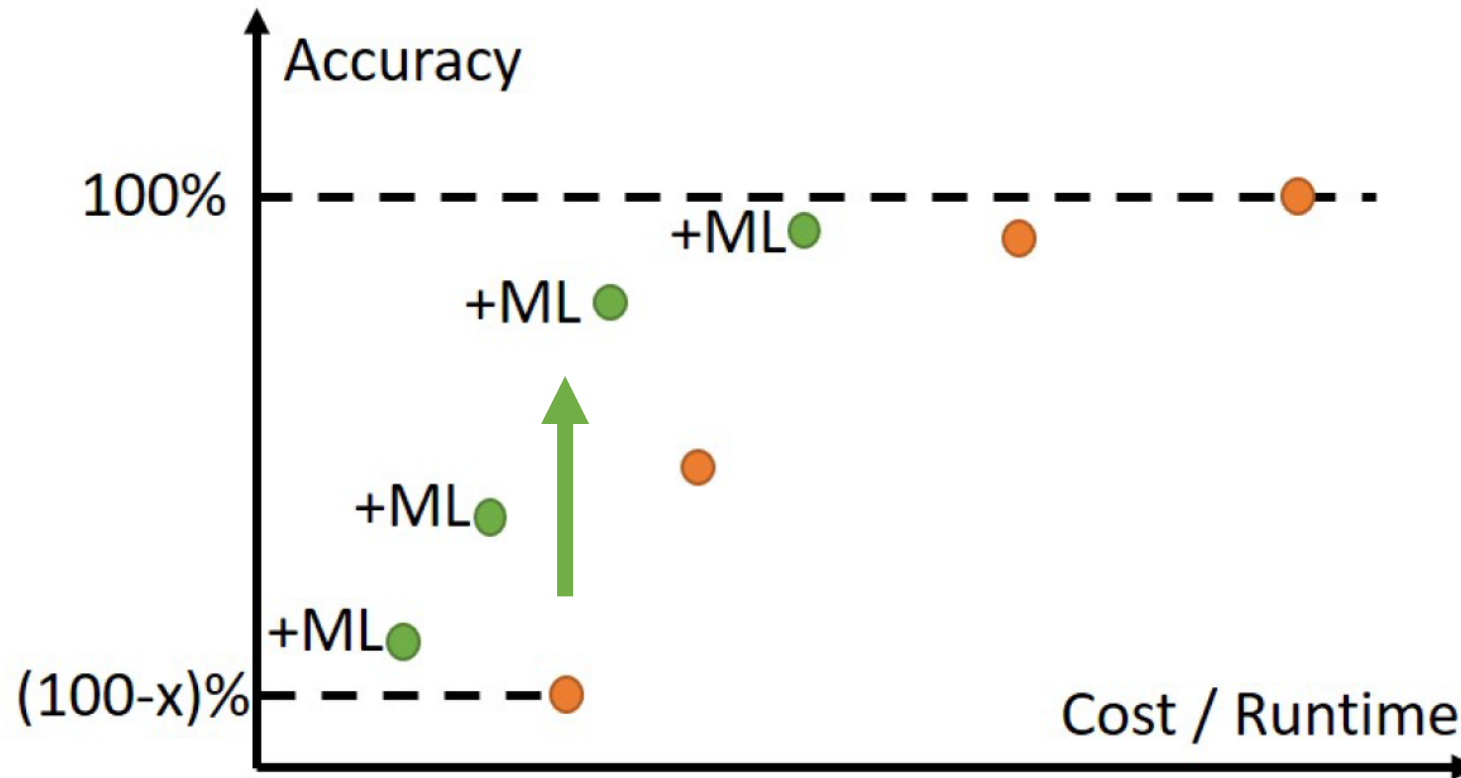
Strategy Card Saves Time, \$



- **TYPE 1 Prediction Error:** MDP STOPs a run that will eventually succeed
- **TYPE 2 Prediction Error:** MDP predicts GO at each iteration, but run fails

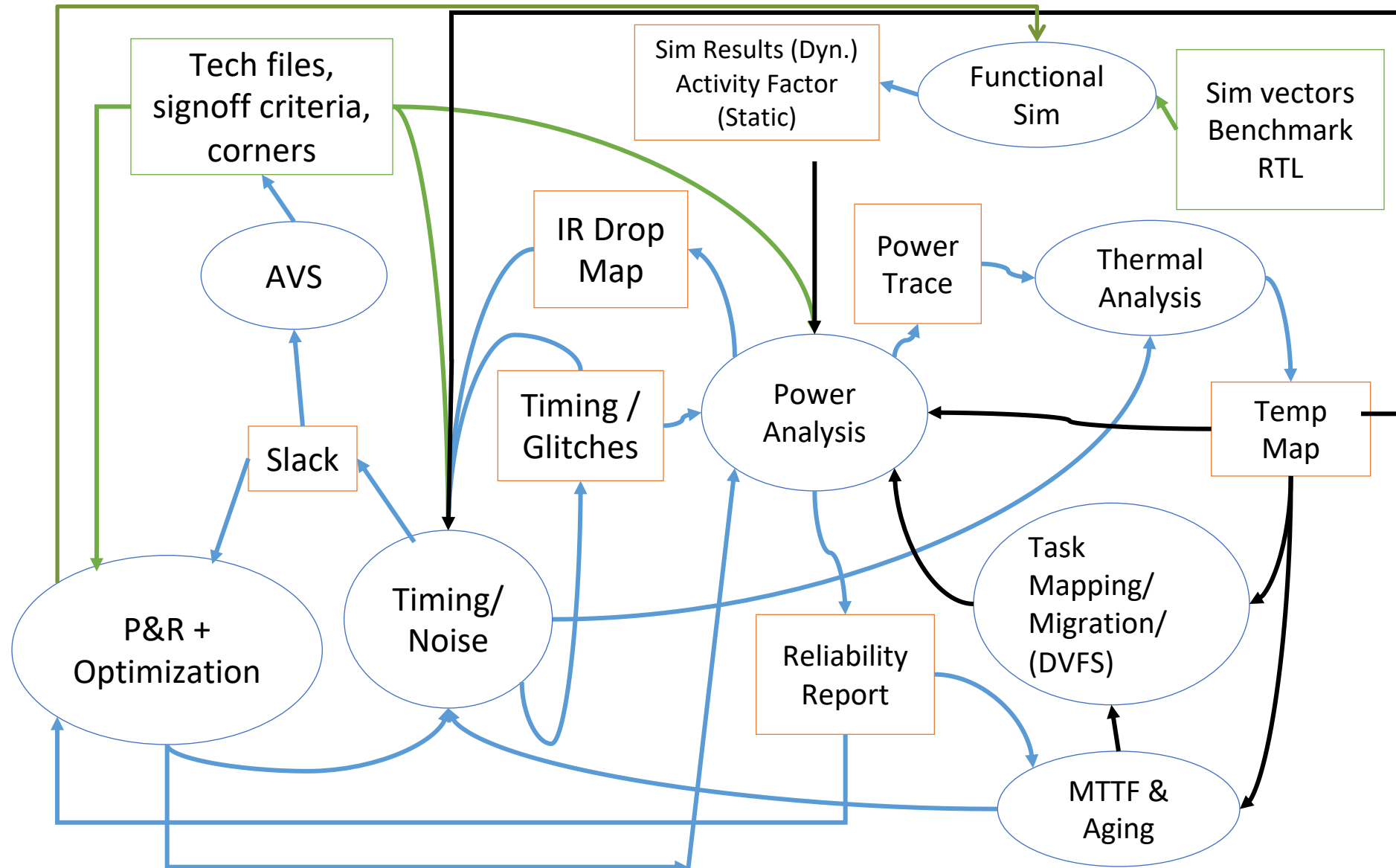
Errors	Training (1200 logfiles)			Testing (3442 logfiles)		
N = 200	Training Error	#TYPE 1	#TYPE 2	Testing Error	#TYPE 1	#TYPE 2
1 STOP	29.17%	251	99	38.3%	1317	3
2 consec STOPs	10.5%	27	99	9.0%	307	3
3 consec STOPs	8.5%	3	99	4.6%	154	3

No-Brainer: Shift Accuracy-Cost Tradeoffs



Tagline: “It’s Just Physics!”

≈ Shortcut (Multiphysics! Workloads!) Analysis Loops?



“Target List”: Predictive Models of Tools, Designs unchanged since 2018

- Predict convergence point for P&R, non-uniform PDN
- Estimate PPA response of block to floorplan context
- Estimate useful skew impact on post-route WNS,TNS
- “Auto-magic” determination of netlist constraints for given performance and power targets
- **Key opportunity: exactly ONE netlist is passed into place-and-route – how to generate this best netlist?**
- Predict best “target sequence” of constraints through layout optimization phases = *steering*
- Predict “most-optimizable” cells during design closure
- Predict divergence (detouring, timing/slew violations) between trial/global route and final detailed route
- Predict “doomed runs” at all steps of design flow

Leakage opt, retiming
impact, buffering shocks, ...

ML Key Idea #1: “Conditioning at Interstices”

- **EDA algorithms** are heuristics – **start from blank state**
 - Lacks the ability to learn from previous designs or runs
 - ML can exploit historical data to warm-start and guide search
- **PD flow is siloed** – stages **run independently** with little cross-stage coordination
 - Requires **human expert intervention** at handoff points to recover PPA
 - Can not reason about downstream impact of upstream choices

Co-optimizations

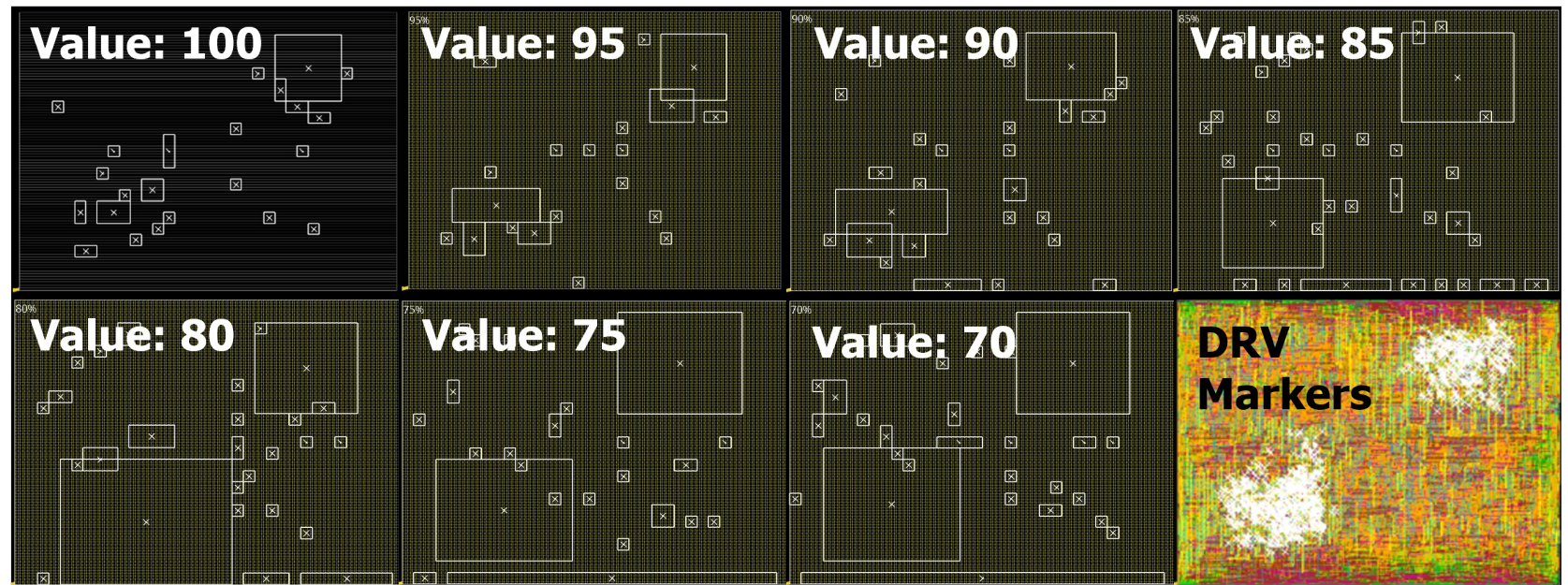
- Netlist – Backend
- Hierarchy – Floorplan
- Floorplan – SP&R
- Synthesis – P&R
- Place – Route
- GRoute – DRoute

“Magic”

- Netlist
- Netlist Partitioning
- Block shaping + *boundaries*
- Placement screens
- Route screens
- Route guides
- Corners + *endpoint SDCs*
- Constraints
- Tool/engine recipes
- ...

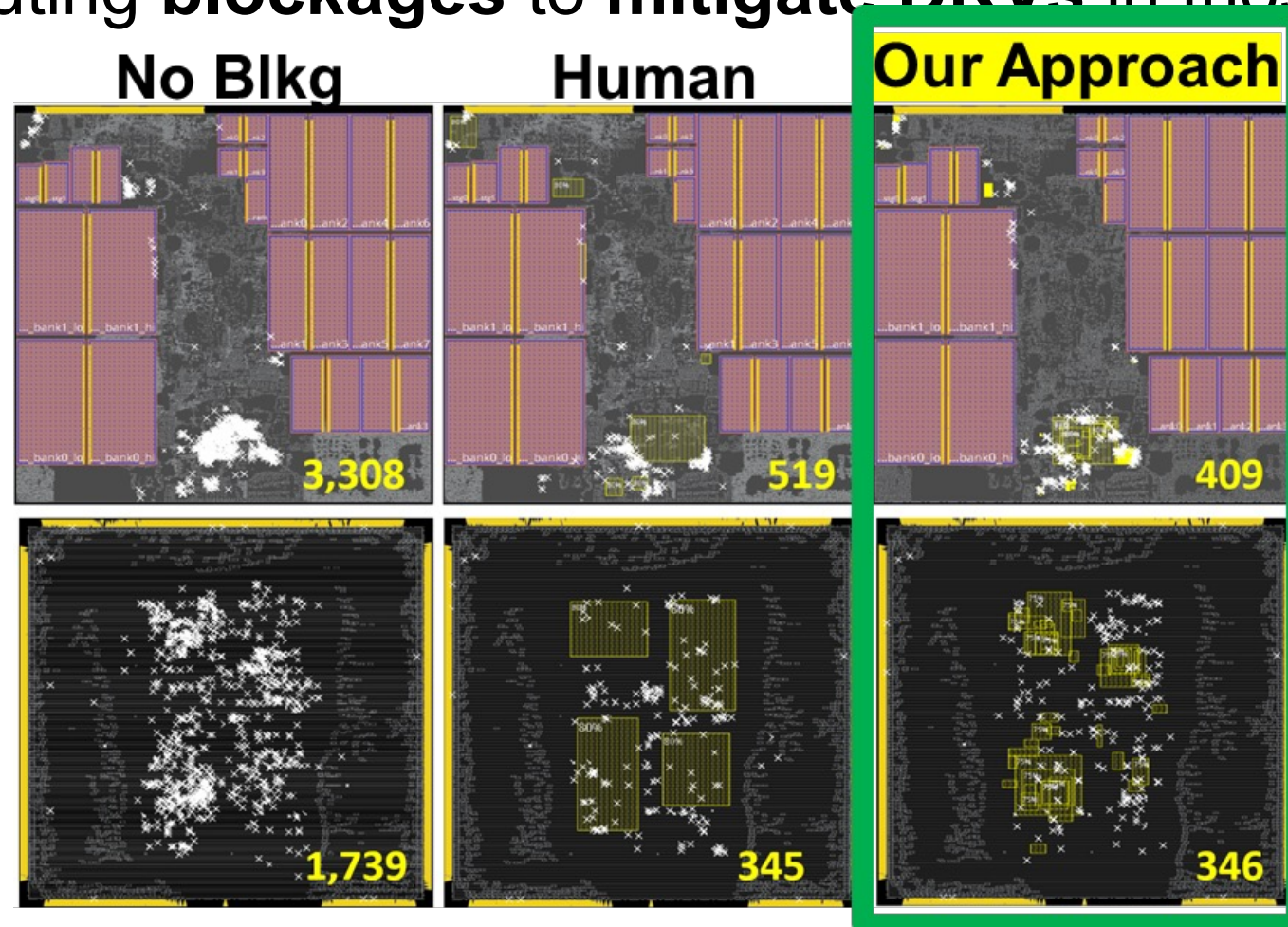
ML Key Idea #2: Take Multiple Views in Unit Time: Tomography

- Congestion report using Commercial Tool
 - Varying partial density value of the routing blockage from 70 – 100
 - Union of congestion reports enables better ML-based alignment with actual DRC markers
 - Partial density = 75 is closest match to actual markers, in this case
- Routing runtime = 1.5 hours; “fast GR” runtime < 1 second



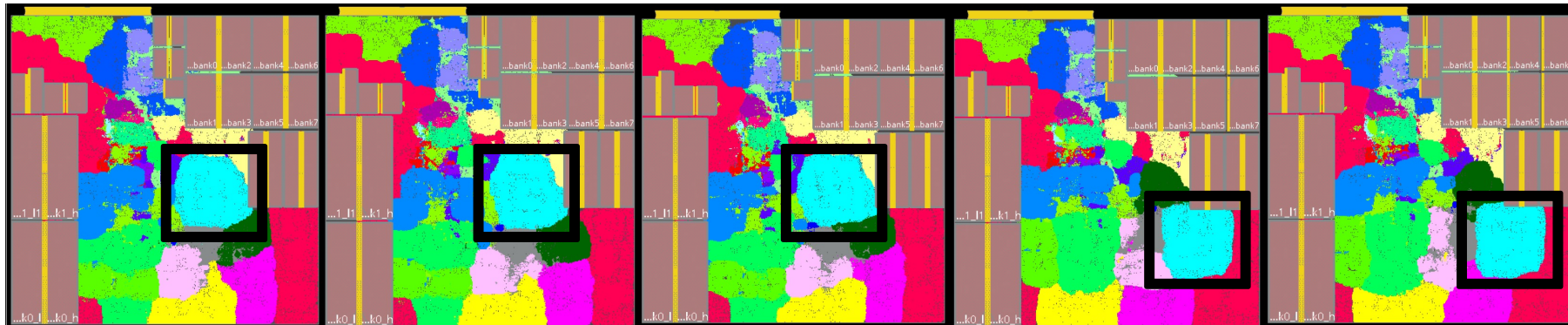
Placement Tomography = condition the routing problem to improve changes of success !

- Use tomography features of a given placement to
 - **Predict layer-wise DRV hotspots**
 - **Generate routing blockages** to mitigate DRVs in those hotspots



From Placement Tomography to **Netlist** Tomography

- **Soft guides:** soft placement constraints that cluster cells without fixing exact locations → a key **lever** to **improve timing, power, routing**
- **Netlist tomography:** generate multiple netlist views by perturbing the floorplan and running placeOpt for each view
 - Perturb **height** (± 1 row), **width** (± 2 sites), row **orientation** (R0/MX) → 30 views
 - **Extract per-view:** instance locations, edge slack, co-location graphs
 - Instances **consistently co-located** across views → **strong soft guide candidates**
 - PlaceOpt is cheap; 30 parallel runs add modest wall time to the overall flow



Five perturbed floorplan placements showing varying logical hierarchy locations

Placement Tomography: AI/ML Problem Statement

Given

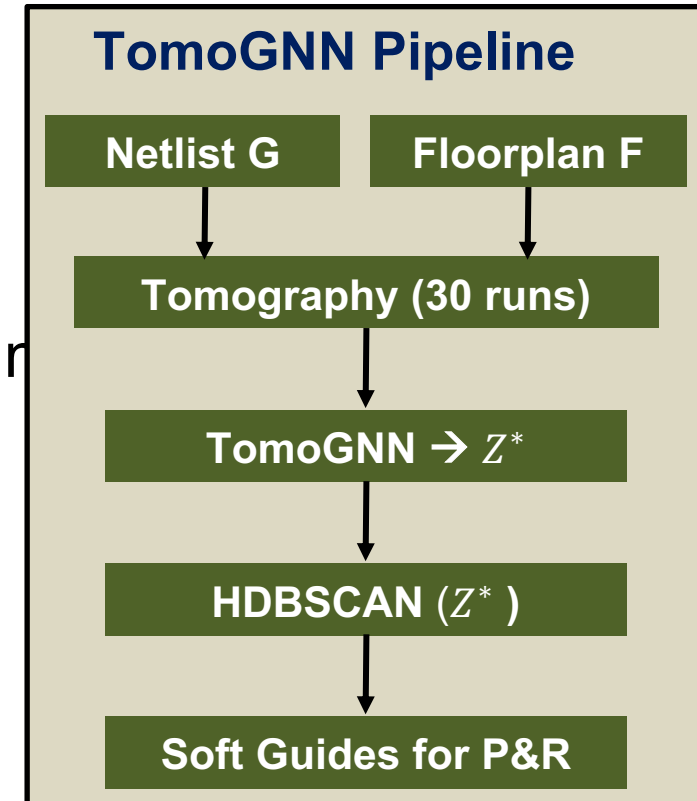
- Synthesized netlist $G = (V, E)$
- Target floorplan F

Goal

- **Learn** embeddings that group timing-critical, spatially proximate instances
- Cluster embeddings $\rightarrow$ soft guides for P&R

Learned clusters encode:

- **Timing criticality:** pulls critical-path instances together
- **Spatial proximity:** respects spatial co-location across views
- **Logical hierarchy:** prefers co-location of same-hierarchy instances



Agenda

- **Old School**

- Predict: Will RouteOpt finish with clean signoff, <1000 DRVs by tomorrow night?
- Classify: Out of these 50 floorplans + budgets, which 3 should go into trial SP&R?
- Estimate: How many hold buffers will the tool add into this post-CTS layout?
- Guide: What P&R tool setup/script will obtain the best QOR within next 36 hours?

- **Agentic Era**

- Challenges #3: validation + evaluation; IP + creators' rights; ecosystem + governance
- Safety
- “Surface” to reason/act over
- Feasibility
- Provenance
- “**Make OpenROAD faster without changing its outputs**”
- “**Summarize this PR and respond as a maintainer would**”

Challenge 1: validation & evaluation

- **Agents are non-deterministic; flows are slow and expensive**

- Reproducibility & traceability – same prompt, different code
- Cost: minutes to hours per RTL-to-GDS candidate
- Improvements at GR / placement may only show after detailed routing

- **What “correct” means is no longer obvious**

- DRC-clean is necessary; not sufficient
- Multi-objective: WL, timing, power, area, runtime
- Generalization across designs and PDKs

- **What we need**

- **Standardized testbeds**: ORFS, MacroPlacement, etc.
- **Hard gates**: DRC, build, regression tests – reject before scoring

- **Formal contracts where possible**: Lean / SMT / static analysis

provable bounds on heuristics?

- **Statistical rigor**: seed sweeps, confidence intervals, ablations

Challenge 3: project & ecosystem governance

- **Agents change the contribution model**

- PRs at machine speed: **how does review scale?**
- Bot accounts, auto-generated commits → **what counts as a maintainer signal?**
- **Risks:** hallucinated, or subtly-wrong code overwhelming reviewers

- **Ecosystem-level questions**

- **Reproducibility:** pin commits, PDKs, agents, prompts
- **Saturated benchmarks:** need open, evolving evaluation suites
- **Compute access:** who can run repo-scale evolution? ← **equity in research**
- **Co-evolution:** agent capabilities and tool APIs both **moving targets**

- **In an ideal (OpenROAD) future ...**

Note: <https://github.com/ieee-ceda-datc/OpenROAD-Research>

- **CI-enforced QoR gates:** non-regressing diffs, machine-checkable invariants
- **Provenance metadata:** model, prompt, evaluation context per diff **else, hallucinations ...**
- **Versioned manifests:** OpenROAD commit + ORFS + PDK + agent → **replay any result**

Toward an open vertical environment for EDA agents

Environment scaffolding (The substrate agents act inside)	Agent capabilities (What we want agents to do)	Ecosystem and governance (What OpenROAD can achieve)
Episode schema for EDA (workspace, actions, constraints, QoR)	Lexicographic, constraint-aware rewards (hard gates → QoR frontier → cost)	Open evolving agent benchmark suite
Cross-stage state & observability (RTL ↔ verif ↔ synth ↔ PD ↔ signoff causality)	Cross-tier orchestration (flow-level + code-level in one loop)	Reproducibility manifests (pinned commits, PDKs, agent versions, prompts)
Multi-fidelity ladder (lint → sim → synth → PD → signoff audit)	Trajectory-grounded training + research-grade memory	CI-enforced QoR gates + machine-checkable invariants
Fast learned surrogates as first-class env citizens	Code-level R&D	Industrial calibration loop + compute equity

Agenda

- **Old School**

- Predict: Will RouteOpt finish with clean signoff, <1000 DRVs by tomorrow night?
- Classify: Out of these 50 floorplans + budgets, which 3 should go into trial SP&R?
- Estimate: How many hold buffers will the tool add into this post-CTS layout?
- Guide: What P&R tool setup/script will obtain the best QOR within next 36 hours?

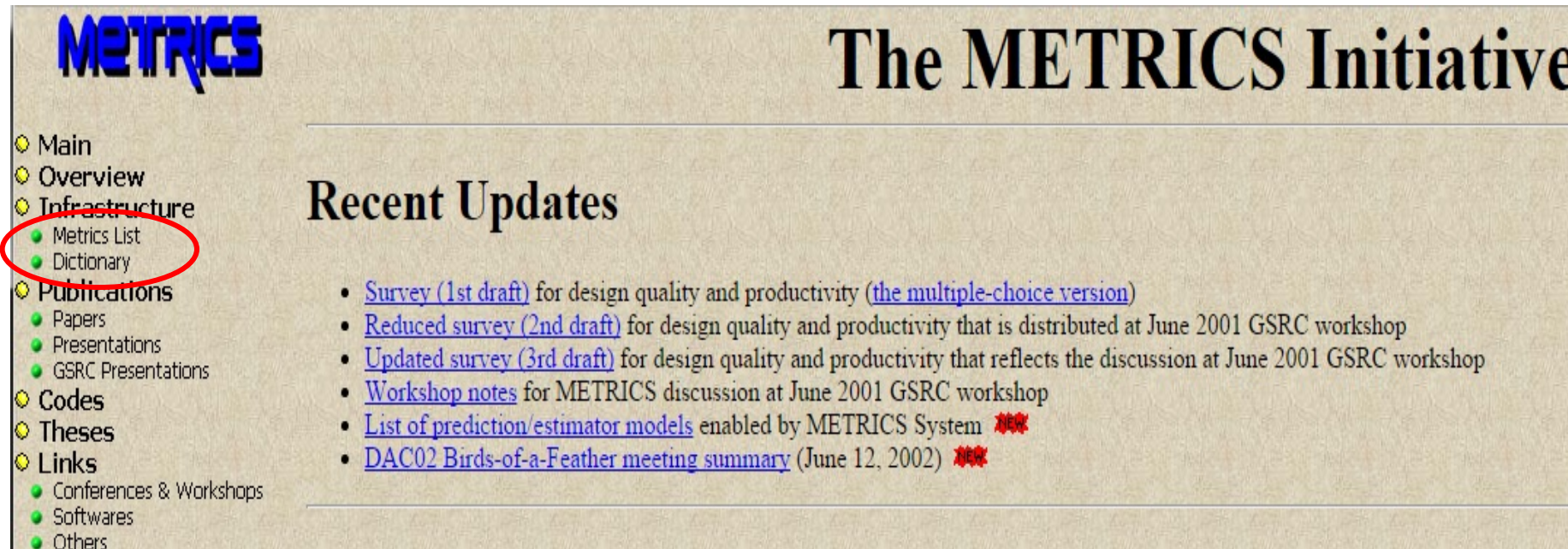
- **Agentic Era**

- Safety
- “Surface” to reason/act over
- Feasibility
- Provenance
- ...
- Challenges: validation + evaluation; IP + creators’ rights; ecosystem + governance

- **Data**

What About: Infrastructure ?

- **Support for ML in IC design ?**
 - Standards for model encapsulation, model application, and IP preservation when models are shared
- **Standard ML platform for EDA tool/flow modeling ?**
 - Enablement of design metrics collection, tool/flow model generation, design-adaptive tool/flow configuration, prediction of tool/flow outcomes
 - Recalls “METRICS” <http://vlsicad.ucsd.edu/GSRC/metrics>
- **More modelable algorithms and tools ?**
 - Smoother, less chaotic outcomes than present methods
- **Datasets to support ML ?**
 - Real designs, Artificial designs and “Eycharts”
 - Shared training data – e.g., analysis correlation (timing, extraction, etc.), post-route DRV prediction, optimal sizing
 - Plus challenges and incentives: “Kaggle for ML in IC design”



- METRICS (1999; DAC00, ISQED01):

“Measure to Improve”

- Goal #1: Predict outcome
- Goal #2: Find sweet spot (field of use) of tool, flow
- Goal #3: Dial in design-specific tool, flow knobs

<http://vlsicad.ucsd.edu/GSRC/metrics>

Challenge 2: humans + agents; IP & creators' rights

- **Where does the human belong in the loop?**

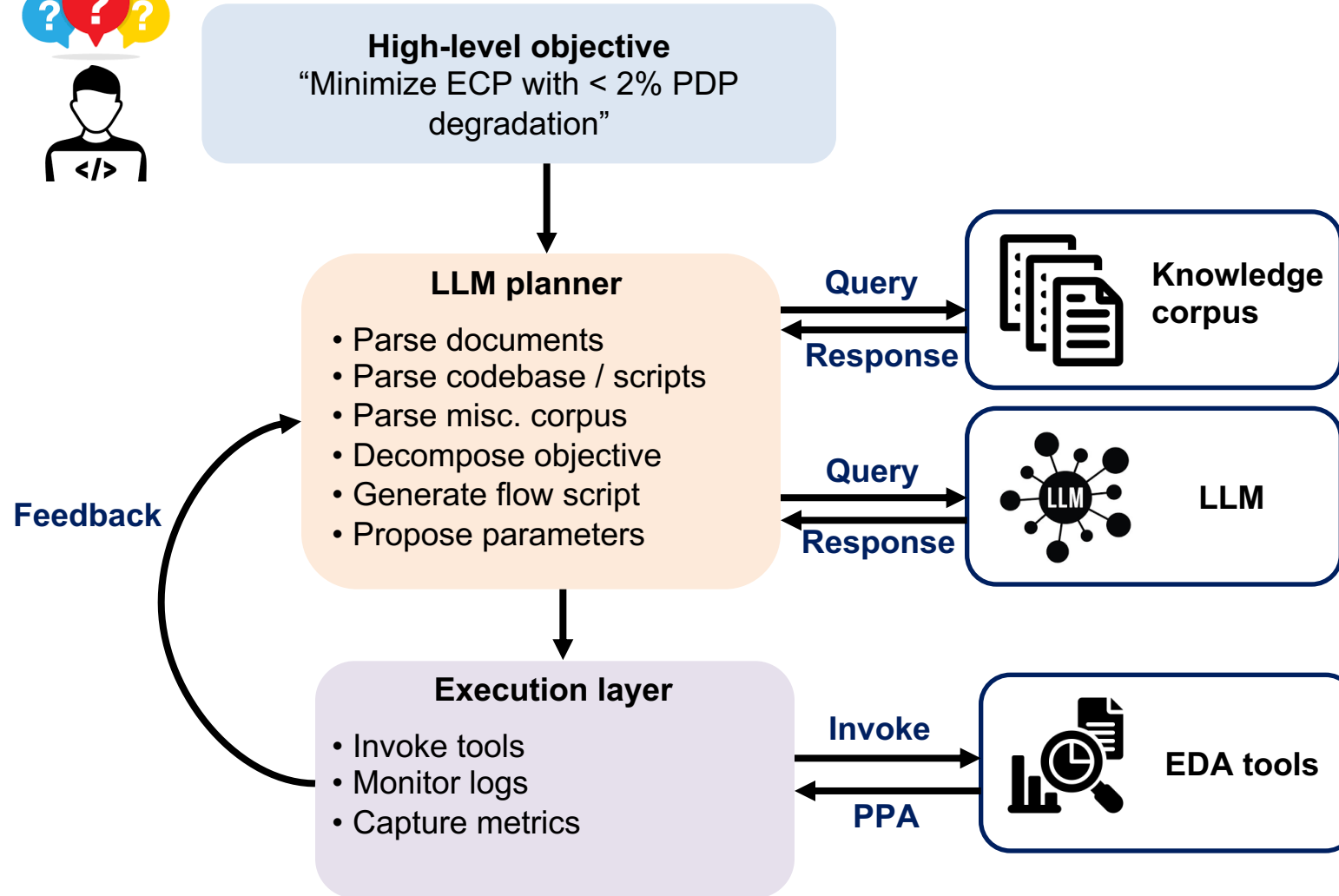
- Objective definition, trade-offs, safety -- not API debugging
- Agents explore; humans judge and are accountable
- **Risk:** **deskilling** if agents become opaque oracle “experts”

- **IP and creators' rights are murky** (*in the ‘agentic future’ ...*)

- **Who owns an LLM-evolved diff?** Model? Prompter? Repo? (*If it reads on a patent claim?*)
- Permissively-licensed code bases might accept agent diffs (*is usability clouded?*)
- **Training** with public “scraped” corpora (*creator rights, opt-ins, ... ?*)
- **Attribution and credit** when an agent rediscovers known techniques

- ***What does “authorship” mean for agent-generated EDA code?***

Flow-level agents – architecture



• ChipNeMo (NVIDIA) (2023)

- Pretraining on expert-engineer scripts
- Combines ICL with document retrieval
- Generates high-quality Tcl scripts for timing analysis, etc.

• ChatEDA (CUHK) (2024)

- Translates flow automation to an agentic task
- Rich training corpora, CoT prompting, usage of multi-agents

• ORAssistant (Precision Innovation) (2024)

- Automated script generation and chatbot for OpenROAD ecosystem

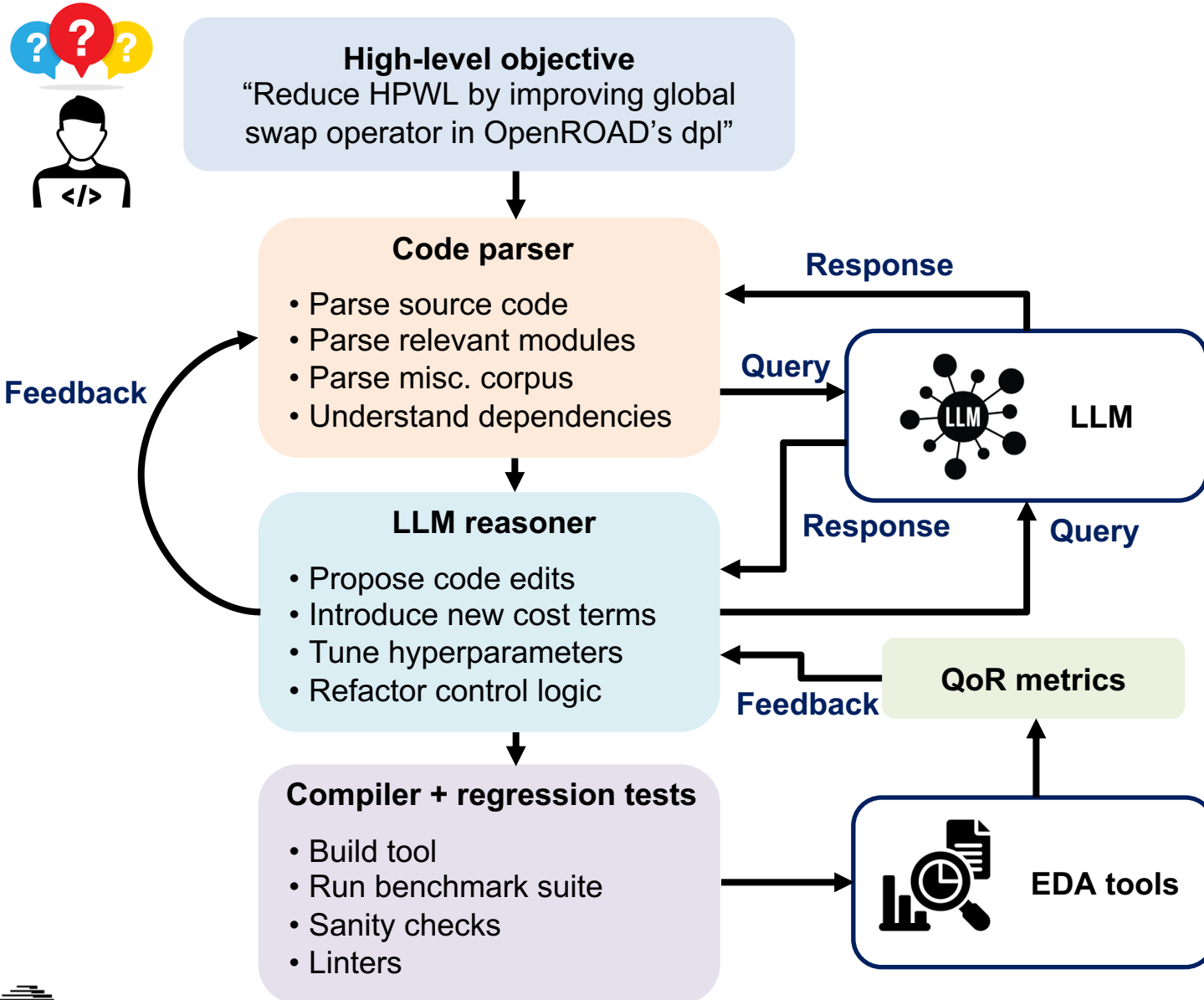
• ORFS-agent (UCSD) (2025)

- LLM-driven iterative optimization agent for parameter tuning in the [OpenROAD-flow-scripts repo](#).

Treat EDA tools as black-boxes; LLM decomposes objective into subtasks; each subtask is translated to scripts



Code-level agents – architecture



• EvoPlace (CUHK) (2025)

- Python code edits inside DREAMPlace framework

• SATLUTION (NVIDIA) (2025)

- C++ code diffs targeting Boolean Satisfiability problems
- Directly evolves solver repositories

• -EVO (UCSD) (2025)

- C++ code diffs targeting placement algorithms in OpenROAD
- Directly evolves placement repositories

• AuDoPEDA (UCSD) (2026)

- Proposes research directions + expands them into executable code diffs

• VLM-MPL (UCSD) (2026)

- Vision language models-based macro placer



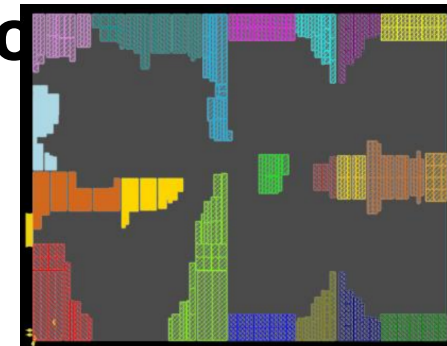
**Modify internal heuristics and algorithms;
LLM reads, proposes and edits code diffs**

Target List: Improving Analysis Correlation

- **Prediction of the worst PBA path**
- **Prediction of the worst PBA slack per endpoint, from GBA analysis**
- **Prediction of timing at “missing corners”**
 - Predict other impacts (e.g., transition times, ..) of an ECO as well
- **Closing of multi-physics analysis loops**
 - Early priorities: vectorless dynamic IR drop, power-temperature loops
- **Continued improvement of timing correlation and estimation !**
 - Faster and better is always helpful !

Target List: “Last Mile” Robots

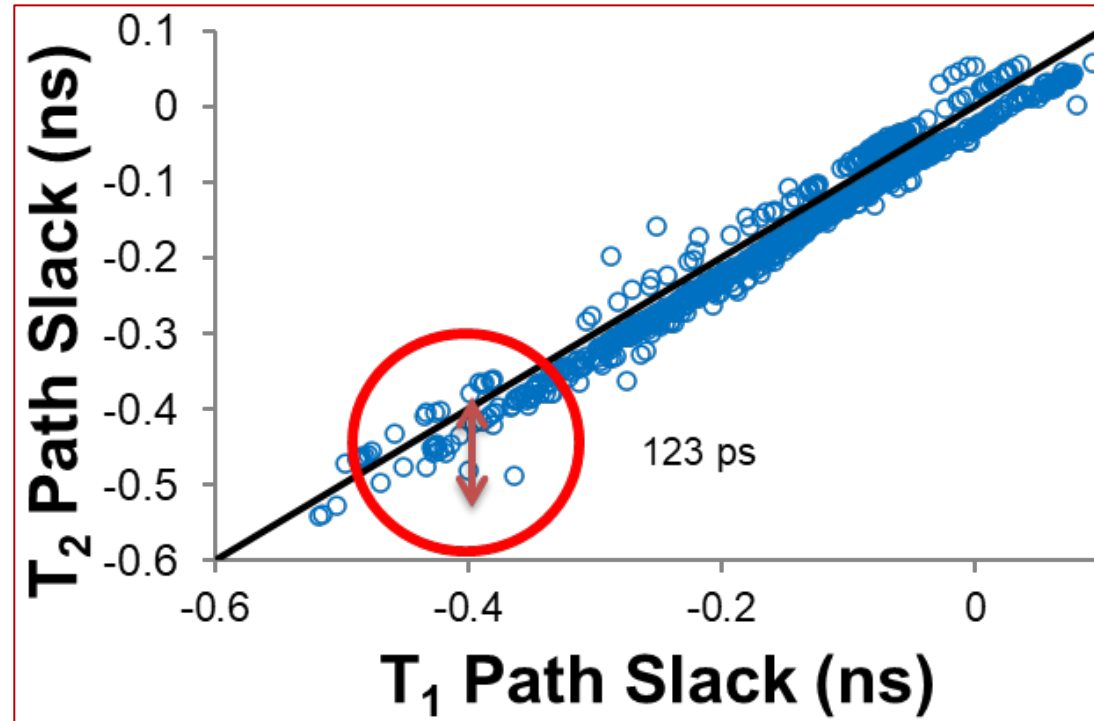
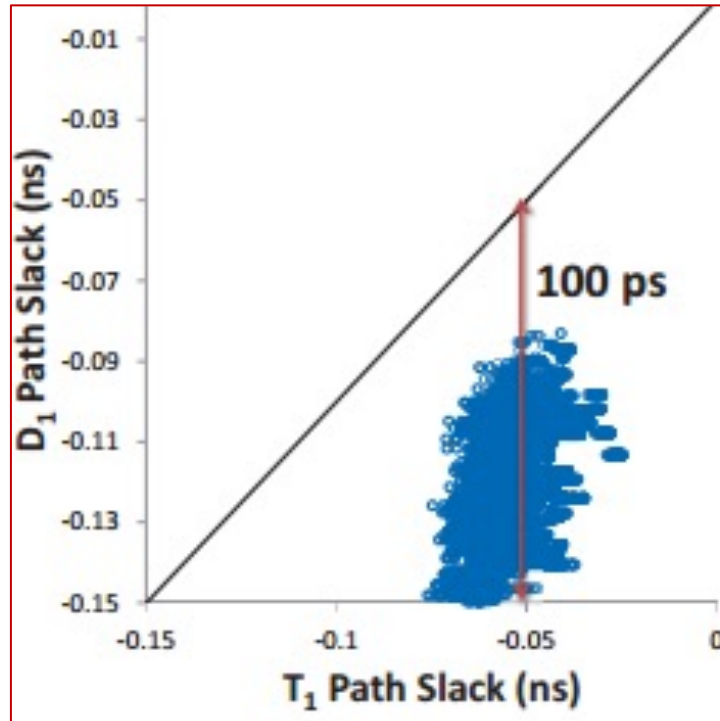
- **Automation of manual DRC violation fixing**
 - P&R tools cannot handle latest rule decks, unavoidable lack of routing resource in high-utilization block, etc.
- **Automation of manual timing closure**
 - After routing and optimization, several thousand violations of maxtrans, setup, hold constraints exist
 - Engineer fixes 200-300 DRVs by hand, per day
- **Placement of memory instances in a P&R block**
 - Reinforcement learning approach: [here](#)
- **Package layout automation**
 - Predict post-routed quality (bump inductances) die-package pin map
- **Required for: pin map, power delivery optimization**



of SoC floorplan +
<http://www.ispd.cc/slides/2018/k1.pdf> , Slide 50

ML to Fix Timing Mismatch

UCSD, [DATE-2014](#)



D = non-signoff timer (e.g., in P&R tool)
T = “golden” timer (e.g., signoff-qualified)

- Can you explain the slack mismatch?
- What is the impact of the mismatch?